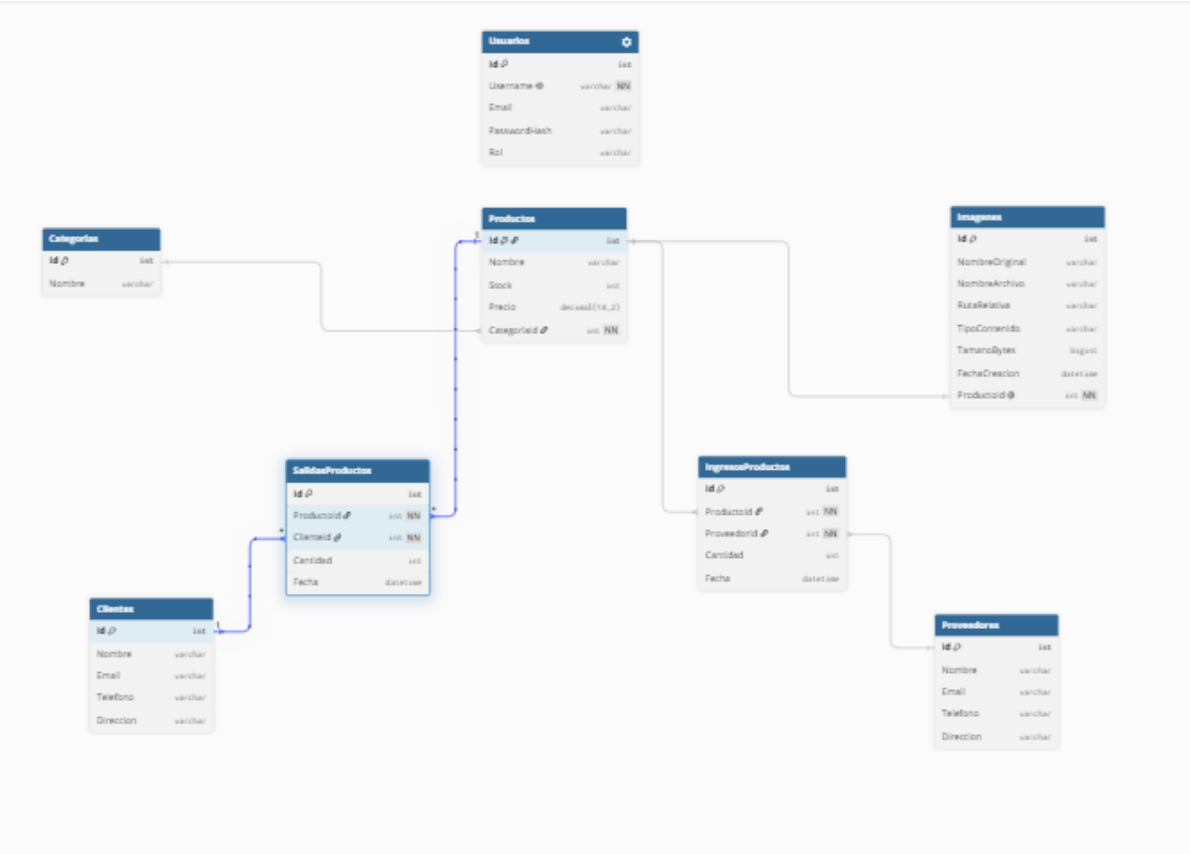
Trabajo Práctico N° 5 - Servicios API REST

DER:

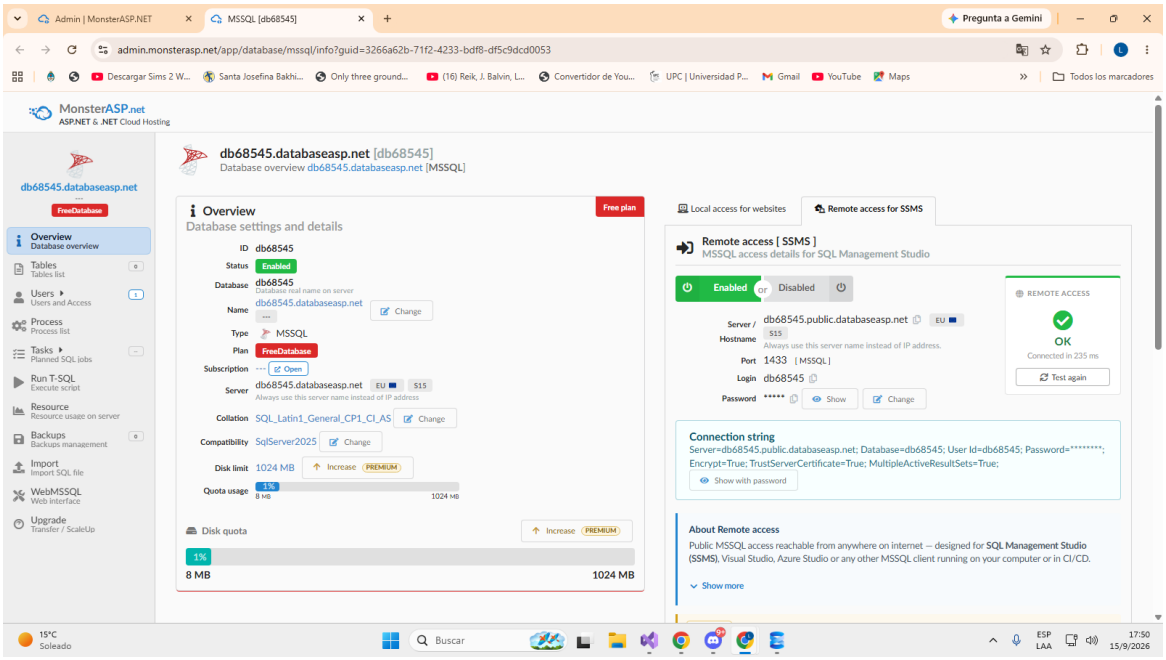


El sistema utiliza una base de datos relacional destinada a la gestión de productos, categorías, usuarios, clientes, proveedores y movimientos de stock.

El diagrama muestra las entidades principales y sus relaciones. Los productos pertenecen a una categoría y pueden tener una imagen asociada. Los ingresos de productos registran el producto, el proveedor, la cantidad y la fecha de ingreso, mientras que las salidas registran el producto vendido, el cliente, la cantidad y la fecha de la operación.

La base de datos está compuesta por las siguientes entidades: **Usuarios, Categorías, Productos, Imágenes, Proveedores, Clientes, IngresosProductos y SalidasProductos**.

Publicación de la base de datos:

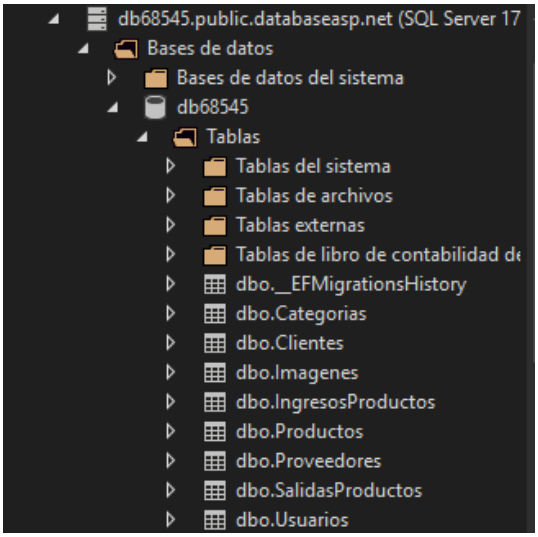


La base de datos relacional utilizada por la API fue publicada en el servicio de hosting **MonsterASP.NET**, utilizando **Microsoft SQL Server**.

En el panel de administración se puede observar la base de datos creada en el servidor y la información correspondiente a su instancia. La API publicada utiliza esta base de datos remota para almacenar y consultar la información del sistema.

La publicación de la base de datos permite que la API funcione de manera remota, sin depender de la instancia local de SQL Server utilizada durante el desarrollo.

Estructura de la base de datos publicada

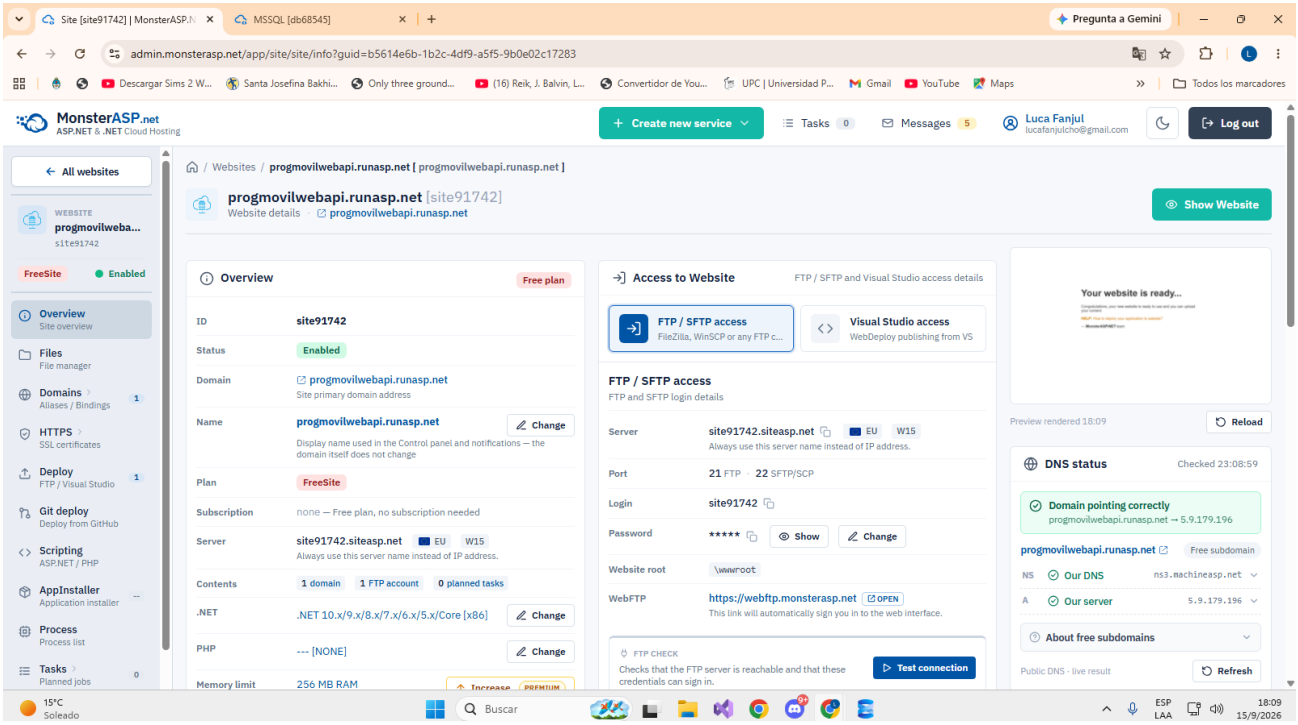


Una vez publicada la base de datos en SQL Server, se verificó su estructura mediante SQL Server Management Studio.

La base db68545 contiene las tablas utilizadas por la API: **Categorias, Clientes, Imagenes, IngresosProductos, Productos, Proveedores, SalidasProductos y Usuarios**.También se encuentra la tabla `__EFMigrationsHistory`, utilizada por Entity Framework Core para registrar las migraciones aplicadas sobre la base de datos.

Esta verificación confirma que el modelo desarrollado localmente fue correctamente generado en la base de datos alojada en el servidor.

Publicación de la API REST:

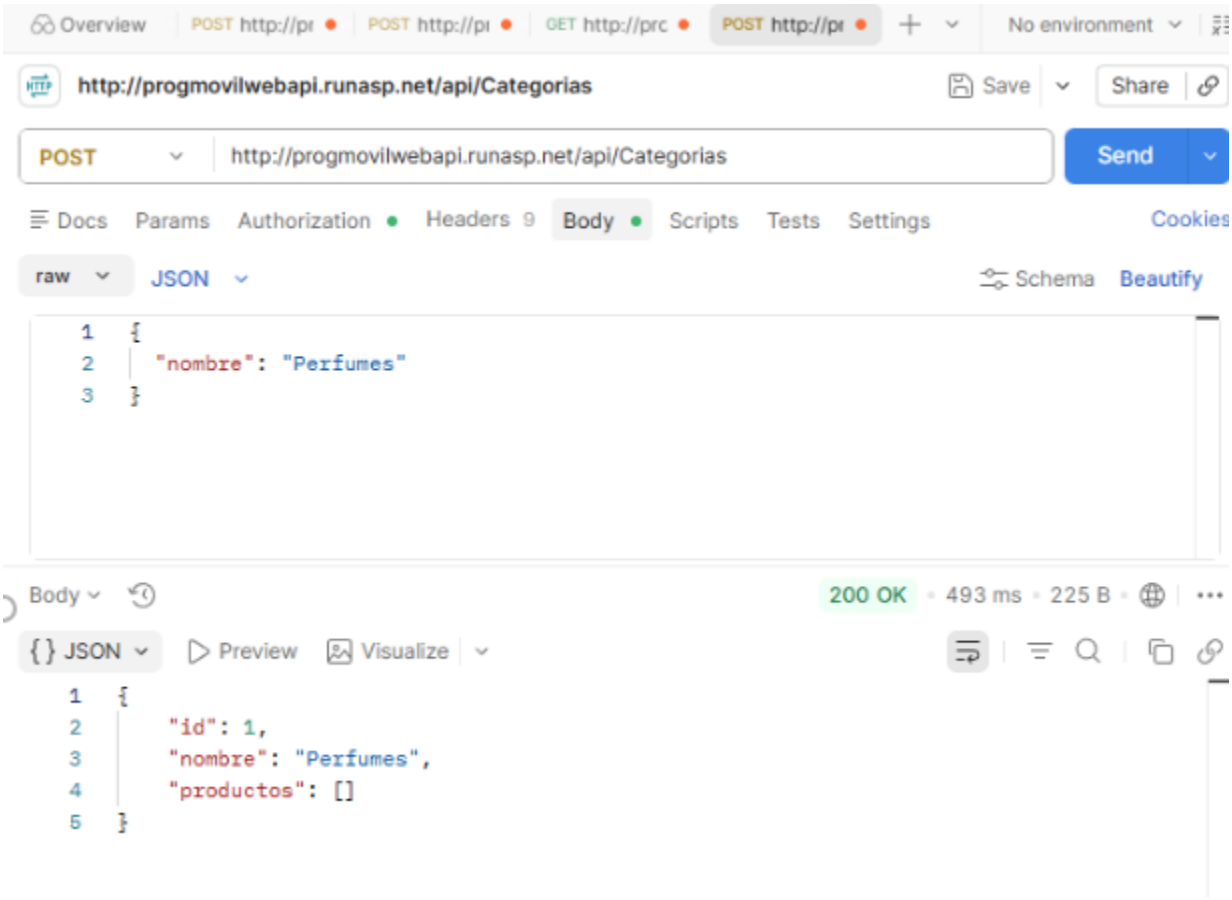


La API REST desarrollada en ASP.NET Core .NET 10 fue publicada en el servicio de hosting MonsterASP.NET. Para ello se creó y configuró un sitio web destinado a ejecutar la aplicación en un entorno remoto.

En el panel de administración se puede observar que el sitio se encuentra **habilitado** y posee un dominio público asignado, permitiendo acceder a los endpoints de la API desde Internet y no únicamente desde el entorno local de desarrollo.

Pruebas de la API mediante Postman:

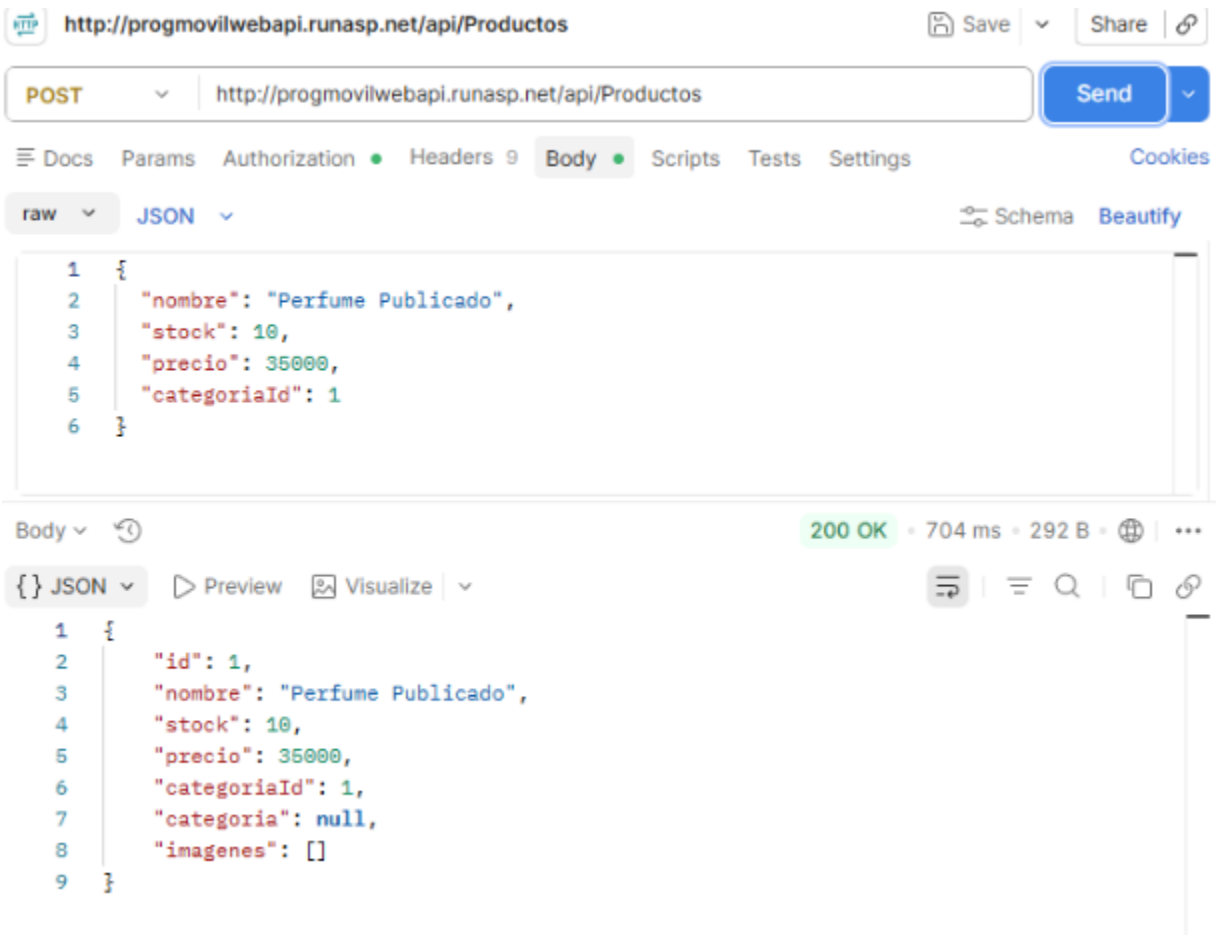
Creación de una categoría:



Se realizó una petición **POST** a `/api/Categorias` para crear una nueva categoría. Para esta prueba se utilizó el nombre **“Perfumes”**.

La API respondió con **200 OK** y devolvió la categoría creada con el `id = 1`, confirmando que los datos fueron registrados correctamente.

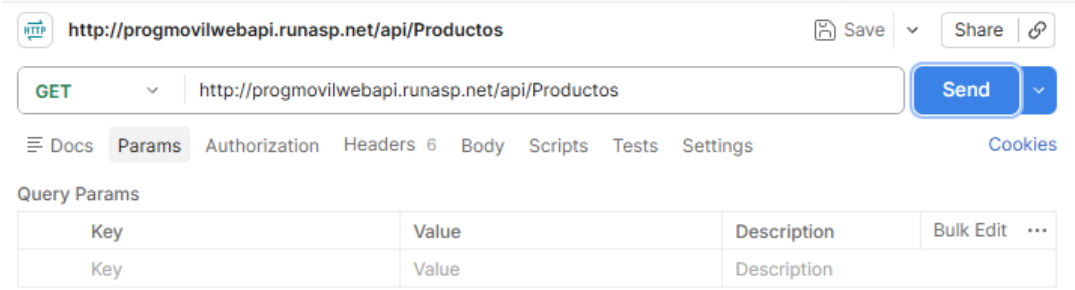
Creación de un producto:

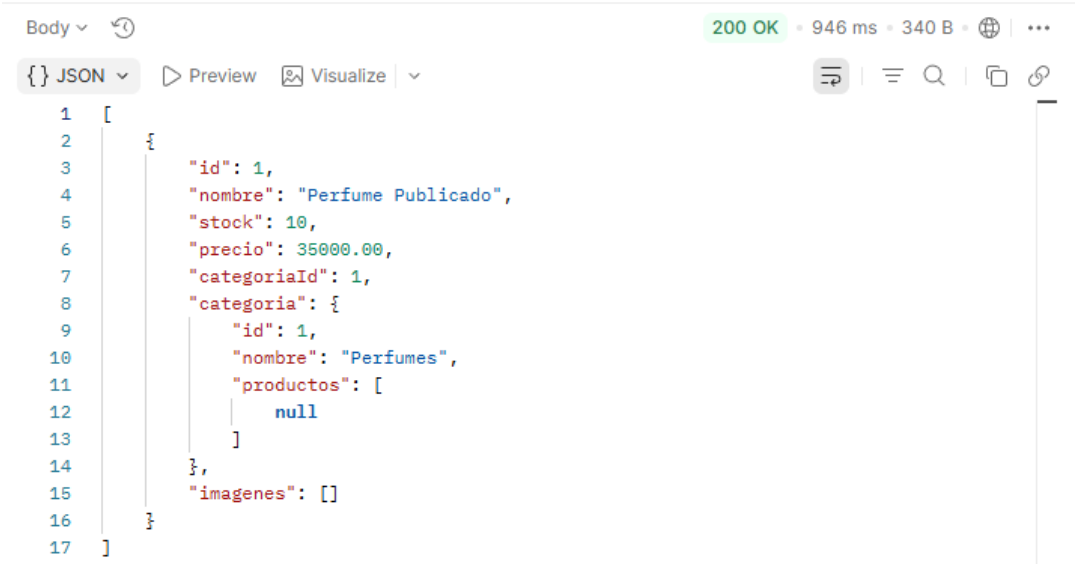


Se realizó una petición **POST** a `/api/Productos` para registrar un producto. Se cargó **"Perfume Publicado"** con un stock inicial de **10 unidades**, un precio de **$35.000** y `categoriaId = 1`, correspondiente a la categoría creada anteriormente.

La API respondió con **200 OK** y devolvió el producto con el `id = 1` asignado.

Consulta del listado de productos:



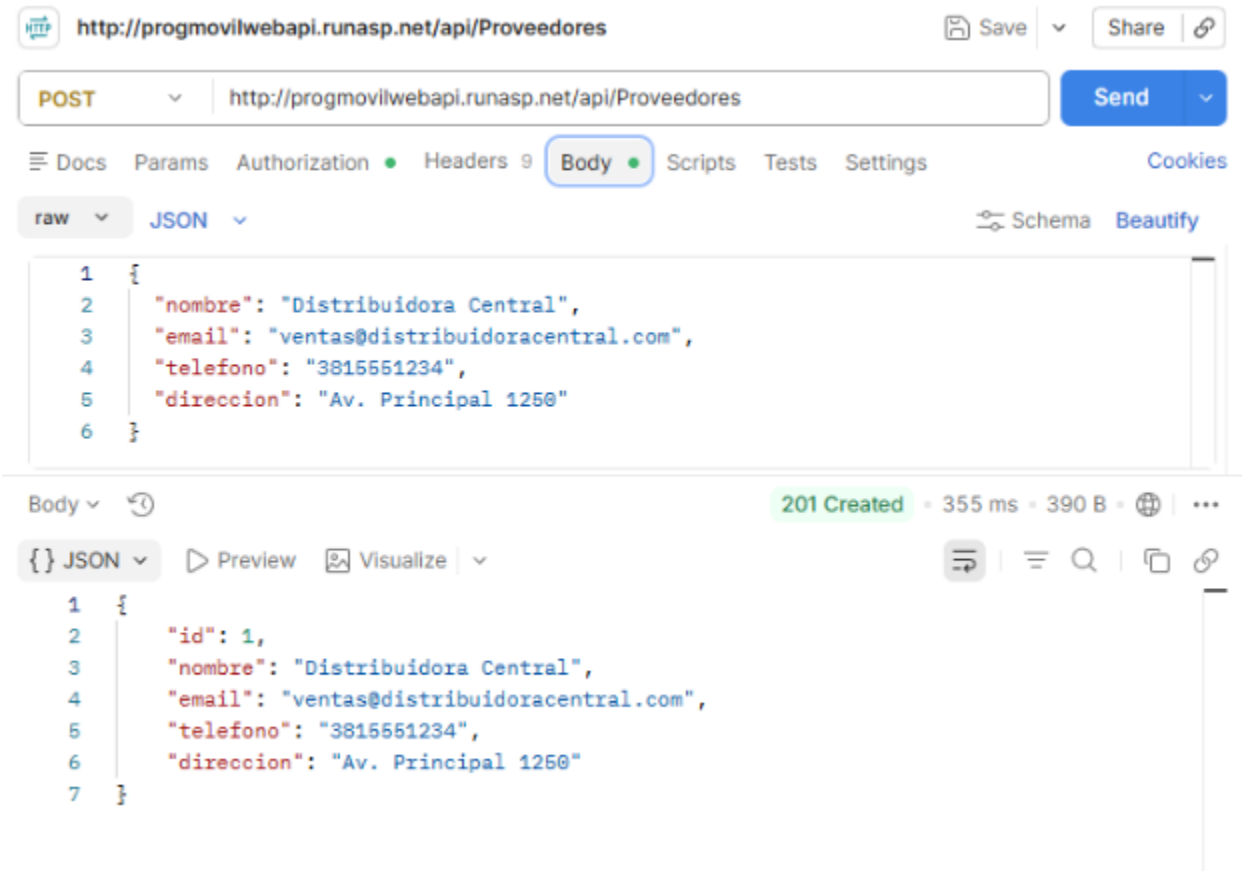


Se realizó una petición **GET** a `/api/Productos` para consultar los productos registrados.

La respuesta fue **200 OK** y se obtuvo el producto **“Perfume Publicado”** creado anteriormente. En el resultado se pueden observar sus datos y la categoría asociada.

Esta prueba también permite comprobar que la información cargada anteriormente quedó almacenada y puede ser consultada desde la API.

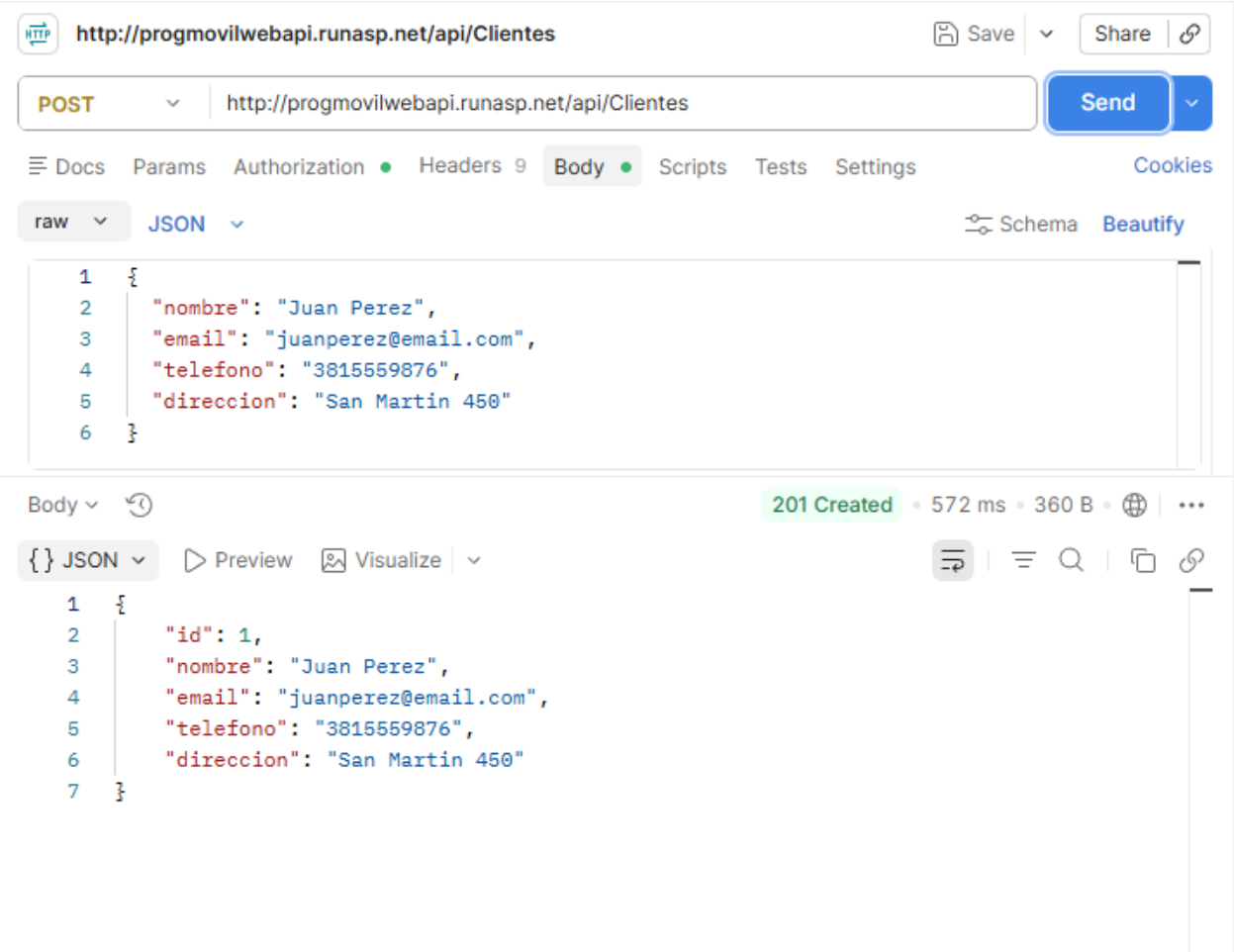
Creación de un proveedor:



Se realizó una petición **POST** a `/api/Proveedores` para registrar un proveedor en el sistema. Para la prueba se cargó **"Distribuidora Central"**, junto con su email, teléfono y dirección.

Como se observa en Postman, la API respondió con **201 Created** y devolvió los datos del proveedor junto con el `id = 1` generado al guardarlo.

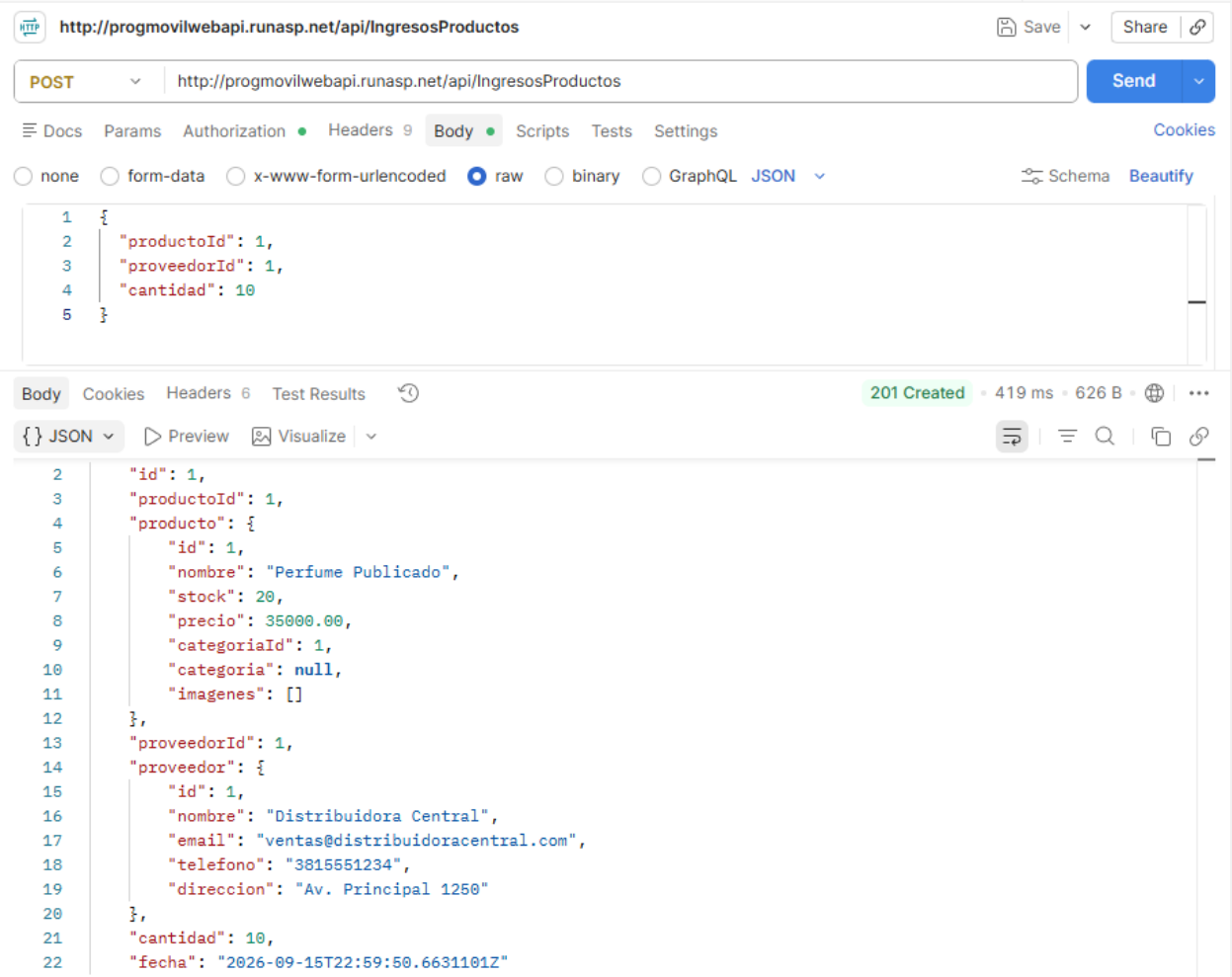
Creación de un cliente:



Se realizó una petición **POST** a `/api/Clientes` para registrar un nuevo cliente. Para la prueba se cargó **"Juan Perez"**, junto con su email, teléfono y dirección.

La API respondió con **201 Created** y devolvió los datos del cliente con el `id = 1` asignado al momento de guardarlo.

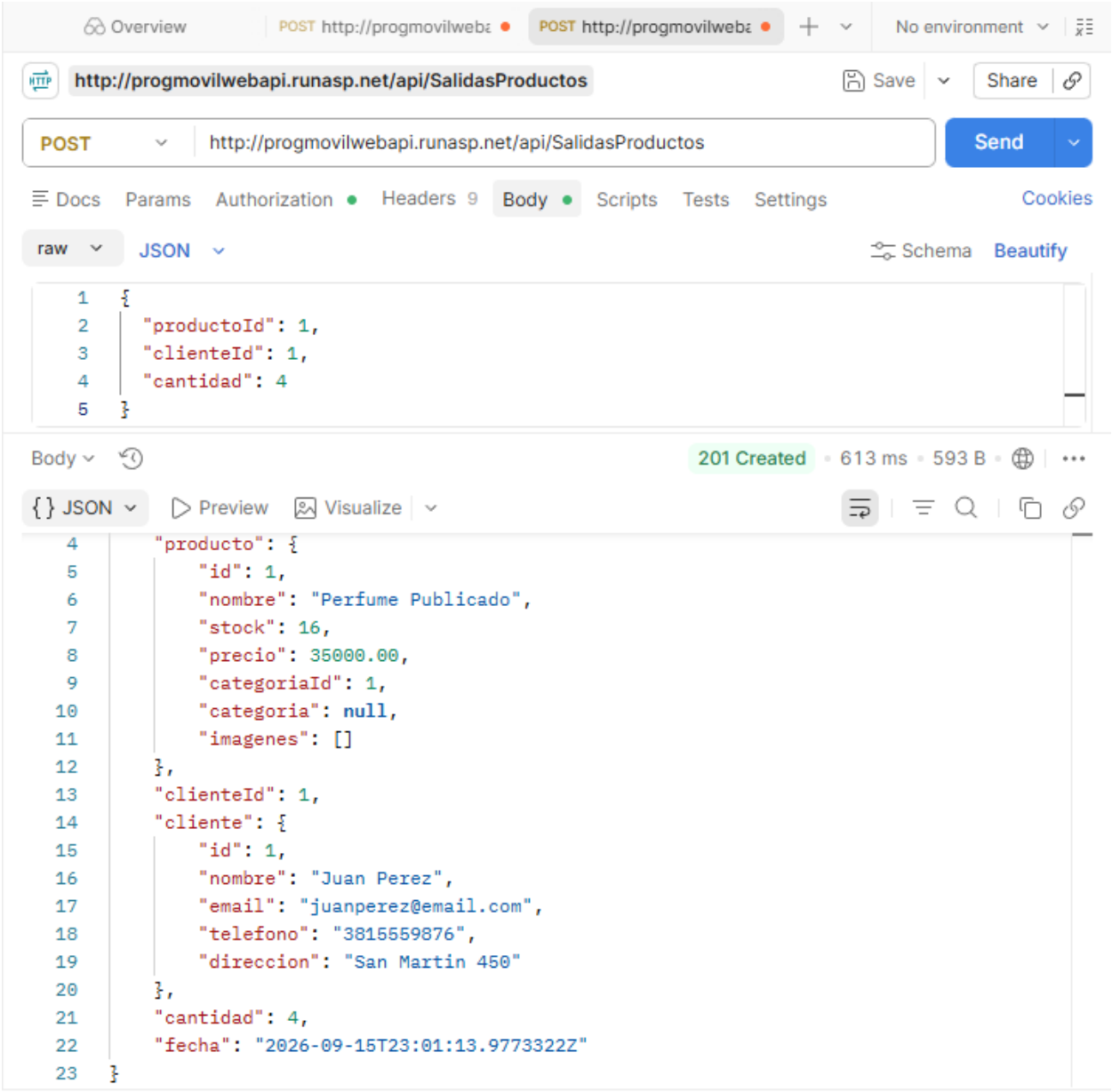
Ingreso de productos y actualización de stock:



Se realizó una petición **POST** a `/api/IngresosProductos` para registrar el ingreso de mercadería. En la prueba se indicó el producto con `productoId = 1`, el proveedor con `proveedorId = 1` y una cantidad de **10 unidades**.

La API respondió con **201 Created** y registró el movimiento junto con su fecha. Además, el stock de **“Perfume Publicado”** pasó de **10 a 20 unidades**, ya que el ingreso actualiza automáticamente el stock del producto.

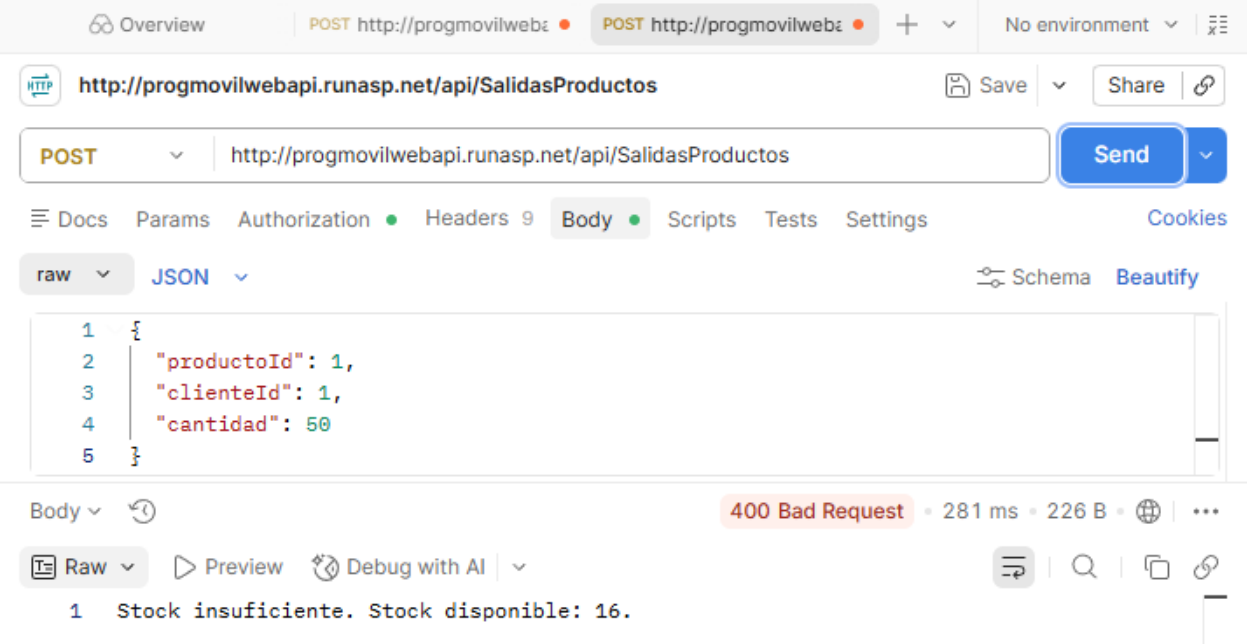
Salida de productos y actualización de stock:



Se realizó una petición **POST** a `/api/SalidasProductos` para registrar la salida de un producto. En la prueba se indicó el producto con `productoId = 1`, el cliente con `clienteId = 1` y una cantidad de **4 unidades**.

La API respondió con **201 Created** y registró la salida junto con su fecha y el cliente asociado. Como resultado de la operación, el stock de **“Perfume Publicado”** disminuyó de **20 a 16 unidades**.

Validación de stock insuficiente:

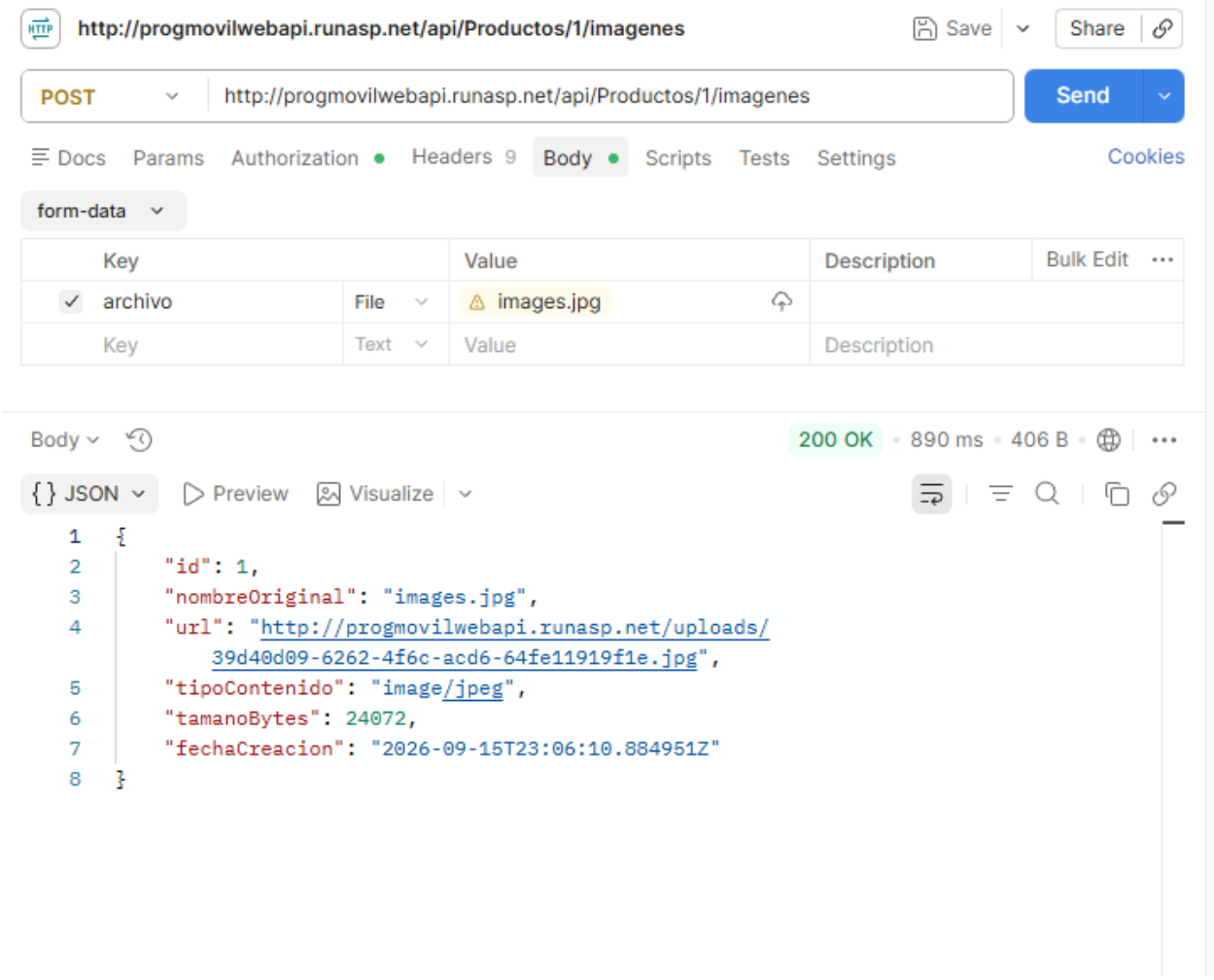


Se realizó una petición **POST** a `/api/SalidasProductos` intentando registrar una salida de **50 unidades** del producto con `productoId = 1`.

En ese momento el producto tenía solamente **16 unidades disponibles**, por lo que la API rechazó la operación y respondió con **400 Bad Request**, indicando el mensaje **"Stock insuficiente. Stock disponible: 16."**

Con esta validación se evita registrar una venta cuando la cantidad solicitada supera el stock disponible.

Carga de imagen de un producto:

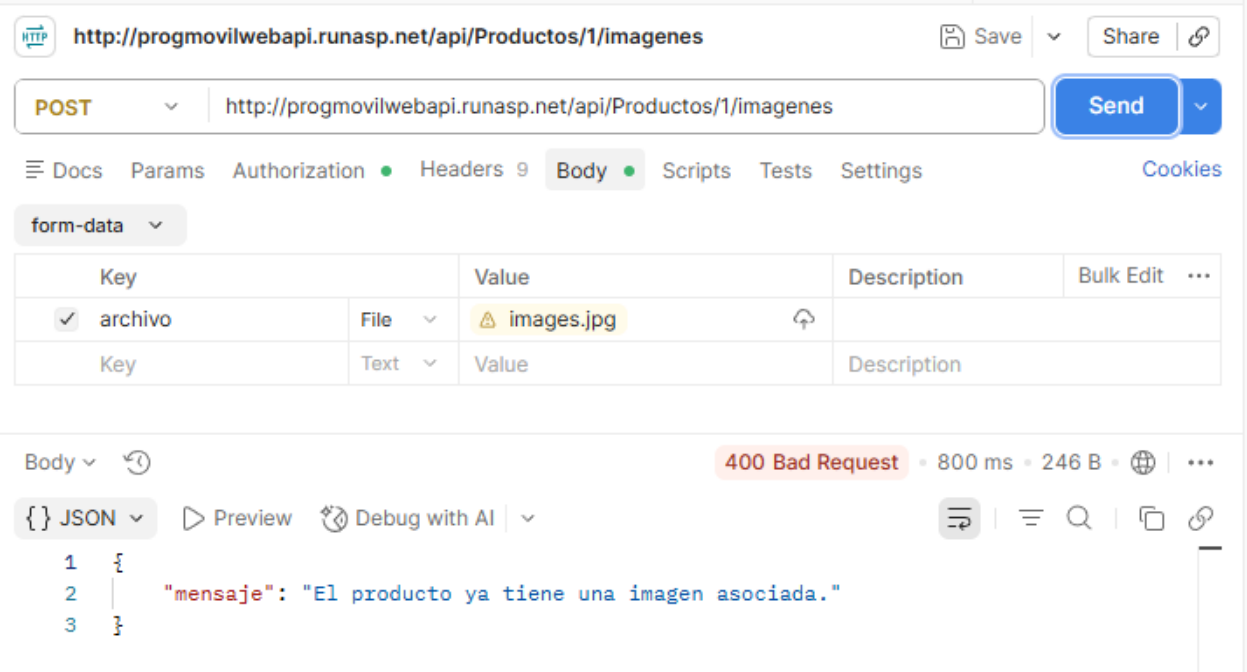


Se realizó una petición **POST** a `/api/Productos/1/imagenes` para asociar una imagen al producto con `id = 1`. El archivo se envió mediante **form-data**, utilizando el campo `archivo`.

La API respondió con **200 OK** y devolvió los datos de la imagen almacenada, entre ellos el nombre original, la URL generada, el tipo de archivo, el tamaño y la fecha de creación.

La imagen queda almacenada en la carpeta **`wwwroot/uploads`** del servidor y su información queda asociada al producto correspondiente.

Validación de una única imagen por producto:

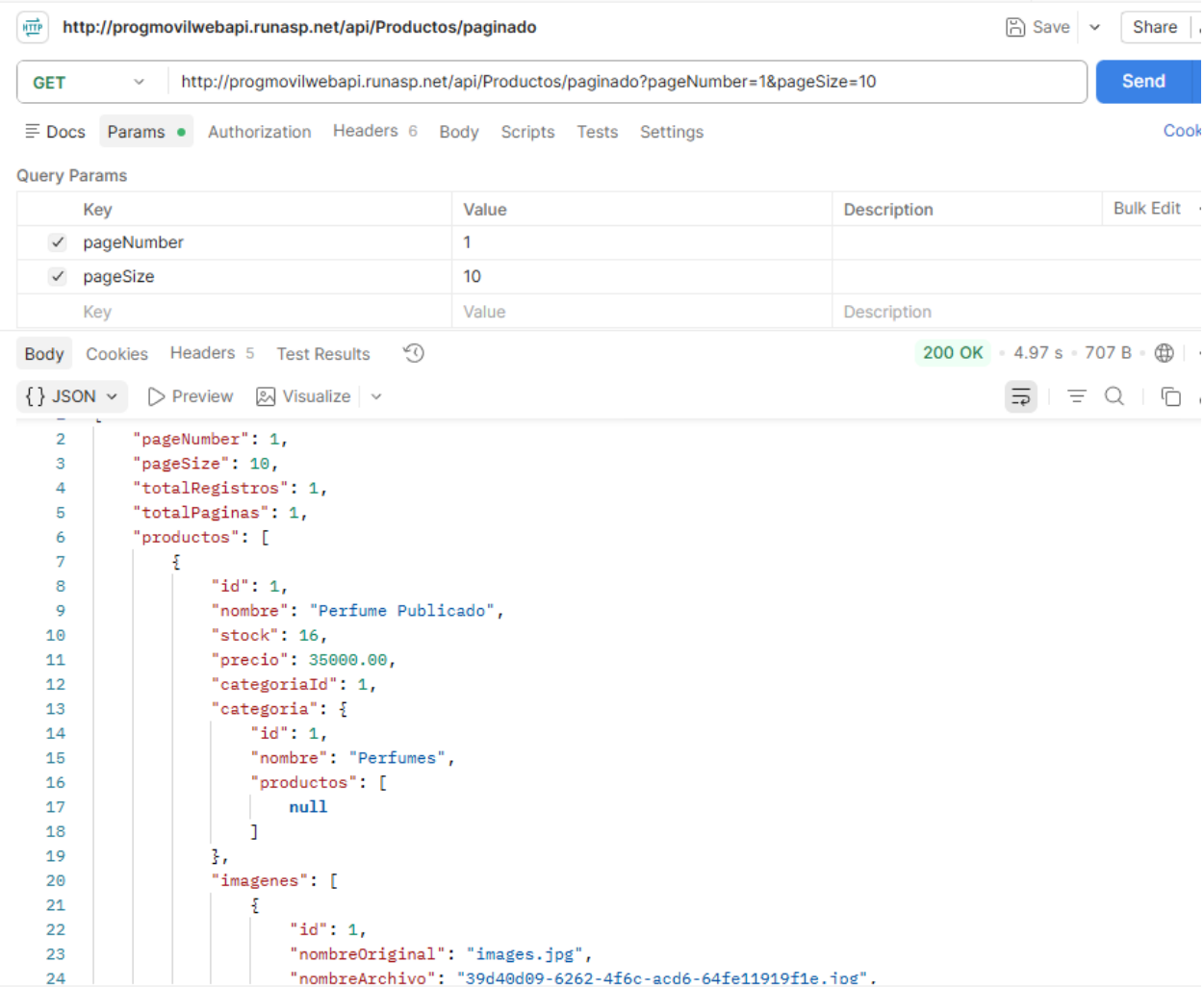


Se realizó nuevamente una petición **POST** a `/api/Productos/1/imagenes`, intentando cargar una segunda imagen al mismo producto.

Como el producto ya tenía una imagen asociada, la API rechazó la operación y respondió con **400 Bad Request**, mostrando el mensaje **“El producto ya tiene una imagen asociada.”**

Esta validación evita que se cargue más de una imagen para un mismo producto, cumpliendo con la restricción definida para el sistema.

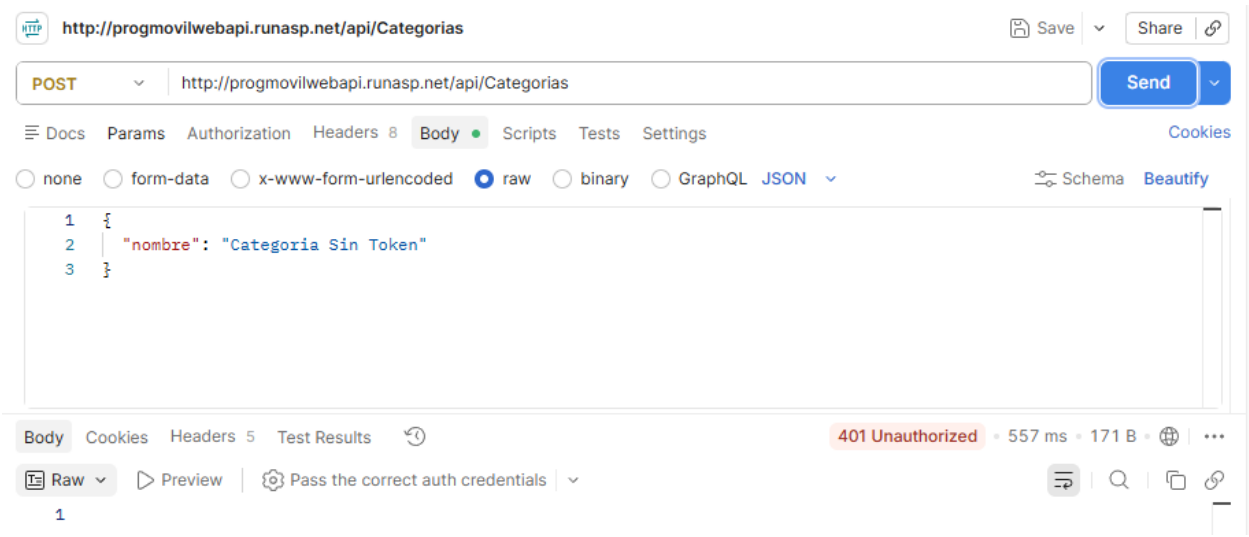
Paginación de productos:



Se realizó una petición **GET** a `/api/Productos/paginado` utilizando los parámetros `pageNumber = 1` y `pageSize = 10`.

La API respondió con **200 OK** y devolvió los productos correspondientes a la página solicitada. Además, la respuesta incluye información sobre la cantidad total de registros y páginas, permitiendo trabajar con listados extensos sin devolver todos los datos en una sola consulta.

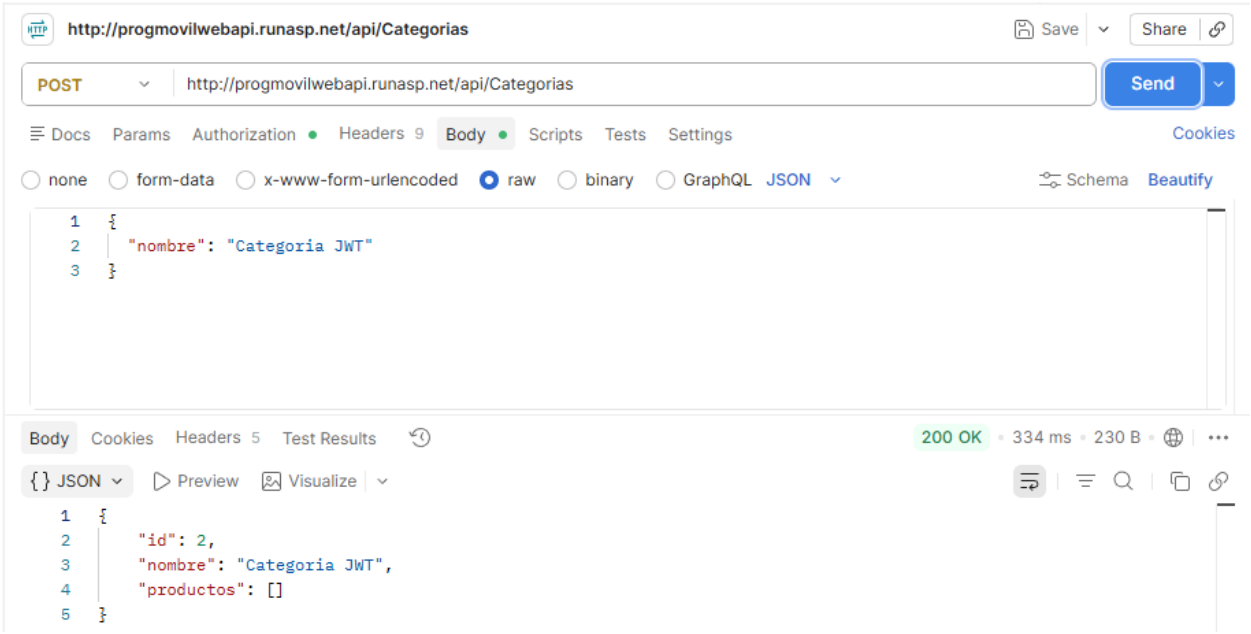
Acceso a ruta protegida sin autenticación:



Se realizó una petición **POST** a `/api/Categorias` sin enviar un token de autenticación.

La API respondió con **401 Unauthorized**, impidiendo realizar la operación. De esta manera se comprueba que las rutas protegidas no pueden ser utilizadas por usuarios que no se encuentren autenticados.

Acceso autorizado mediante JWT:



Se realizó nuevamente una petición **POST** a `/api/Categorias`, esta vez utilizando un **token JWT de un usuario con rol Administrador** mediante autenticación Bearer.

La API validó el token y permitió realizar la operación, respondiendo con **200 OK** y registrando correctamente la nueva categoría. Esto permite comprobar el funcionamiento de la autenticación mediante JWT y la autorización aplicada sobre las rutas protegidas.